KUBERNETES

DAYS 50-55

Complete Beginner-Friendly Learning & Hands-On Practice Notes

THE SIX-DAY LEARNING PATH

50. Cluster & architecture	**51. Pods & YAML**	**52. Namespaces & Deployments**
53. Services & networking	**54. ConfigMaps & Secrets**	**55. Persistent storage**

What this guide gives you

- Plain-English explanations of Kubernetes words, YAML fields, and kubectl commands.
- Step-by-step labs, expected results, checks, cleanup, and troubleshooting.
- PowerShell-friendly instructions for Windows users.
- Practice challenges, revision checklist, and a final mini-project.

Learning Rule

Do not just copy commands. Before running each command, say what you expect it to do. After running it, verify whether Kubernetes did what you expected.

How to use this workbook

Follow the days in order. Each day builds on the previous one. A Kubernetes cluster is your practice environment; YAML files describe the resources you want; kubectl sends instructions to the cluster.

Recommended practice setup

- Use one local Kubernetes environment: Minikube, kind, or Docker Desktop Kubernetes. You do not need all three.
- Install kubectl and confirm that it can communicate with your cluster.
- Use PowerShell on Windows for commands shown outside containers. Commands after `kubectl exec` run inside the Linux container.
- Create a folder for your work, for example `kubernetes-days-50-55`, and save each YAML file with the filename shown.

Your command-reading method

Part	Meaning	Example
`kubectl`	Kubernetes command-line client	`kubectl get pods`
`get`	Read/list resources	`get`
`pods`	The resource type	`pods`
`-n dev`	Use namespace dev	`-n dev`
`-o wide`	Show extra columns	`-o wide`
`-f file.yaml`	Read resource definition from a file	`-f nginx.yaml`

Core mental model: HOW KUBERNETES WORKS AT A HIGH LEVEL



Important: Kubernetes is declarative. You normally describe the result you want (for example, three replicas), and Kubernetes controllers keep working to make the actual cluster match that desired state.

Before you begin: useful checks

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
kubectl config current-context
```

Expected: kubectl prints its client version; cluster-info shows the API endpoint; get nodes shows at least one node in Ready state; current-context names the cluster context currently selected.

DAY 50

Kubernetes Architecture & Cluster Setup

Goal: Understand the parts of a Kubernetes cluster, create or connect to a local cluster, and inspect its health.

1. What is Kubernetes?

Kubernetes (often shortened to K8s) is a platform for managing containerized applications. Instead of manually starting containers on individual machines, you tell Kubernetes what should run. Kubernetes schedules containers, restarts failed workloads, and helps keep the cluster close to the state you declared.

Cluster: a group of machines (physical or virtual) working together. A machine in a cluster is called a **node**. A cluster has a control plane and one or more worker nodes.



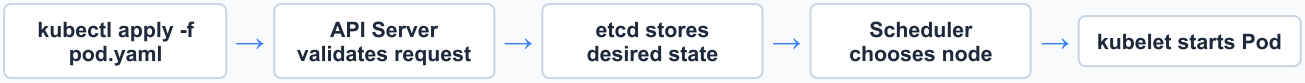
2. Control plane - word-by-word

Component	Plain-English meaning	What it does
API Server	The front door of the cluster	Receives and validates Kubernetes API requests. `kubectl` talks to it.
etcd	The cluster's key-value database	Stores Kubernetes cluster state and configuration.
Scheduler	The Pod placement decision-maker	Chooses a suitable worker node for each unscheduled Pod.
Controller Manager	The group of controllers	Continuously compares actual state with desired state and acts to correct differences.

3. Worker node components

Component	Meaning
kubelet	Agent on a node. Makes sure the Pods assigned to that node are running as requested.
Container runtime	Software that runs containers, commonly containerd or CRI-O.
kube-proxy	Traditionally manages network rules for Service traffic. Some clusters use other implementations.
Pod	Kubernetes' smallest deployable unit. A Pod contains one or more closely related containers.

4. What happens when you apply YAML?



The API Server is the entry point. Controllers and the scheduler help reconcile and place workloads. The kubelet on the chosen node works with the container runtime to start the containers. The Pod becomes Ready only when its containers are running and readiness conditions are satisfied.

5. Set up or connect to a local cluster

Choose one route. Minikube or kind are common local options:

Option A (Minikube):

```
minikube start
kubectl get nodes
minikube status
```

Option B (Kind):

```
kind create cluster --name dev-cluster
kubectl cluster-info
kubectl get nodes
```

6. Explore the cluster & Practice tasks

```
kubectl cluster-info
kubectl get nodes
kubectl describe node <node-name>
```

Practice Tasks: 1. Start local cluster & capture `get nodes`. 2. Find control-plane node & inspect with `describe node`. 3. List namespaces and identify 3 system components. 4. Run `current-context`. 5. Stop and start cluster.

8. Verification and troubleshooting

Symptom	What to check
kubectl: command not found	Install kubectl; reopen terminal; verify PATH.
The connection to the server was refused	Cluster may be stopped; start it and check current context.
Node is Not Ready	Inspect `kubectl describe node`; check local runtime and cluster status.

Day 50 Deliverable: Save notes in `day-50-k8s-setup.md` and include a screenshot of `kubectl get nodes`.

DAY 51

Kubernetes Manifests & Your First Pods

Goal: Create Pods from YAML, inspect them, read logs, enter containers, and use labels.

1. What is a Pod?

A container is a running application process. A Pod is the Kubernetes object that wraps one or more containers and gives them shared networking and storage context. Most beginner workloads use one main application container per Pod.

Analogy: A Pod is like a small workspace. One or more related containers work inside it and share the Pod's network identity.

2. Understand a manifest word by word

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
```

YAML field	Meaning
`apiVersion: v1`	Use the core Kubernetes API version for a Pod.
`kind: Pod`	The type of resource to create.
`metadata / name / labels`	Object identification tags and metadata.
`spec / containers / image`	Desired specification, container definitions, and image to run (pin version for repeatable work).
`containerPort: 80`	Documents ports the container listens on; does not create external access.

3. Create and inspect an Nginx Pod

```
# Save the YAML above as nginx-pod.yaml
kubectl apply -f nginx-pod.yaml
kubectl get pods
```

Expected: The Pod should move from Pending/ContainerCreating to Running. The wide view adds details such as node and Pod IP.

4. Enter the container

```
kubectl exec -it nginx-pod -- /bin/bash
# inside the container:
curl http://localhost:80
```

If the image does not include bash, try `/bin/sh`. The double dash separates kubectl options from the command to run inside the container.

5. Create a BusyBox Pod

```
apiVersion: v1
kind: Pod
metadata:
```

BusyBox is a tiny utility image. The command prints a message and sleeps for one hour to keep the container alive for inspection.

6. Imperative commands & generated YAML

```
kubectl run redis-pod --image=redis:7
kubectl get pod redis-pod -o yaml
kubectl run test-pod --image=nginx --dry-run=client -o yaml
```

Imperative means direct commands. Declarative means describing resources in YAML. Client dry-run checks locally; server dry-run validates via the API Server.

7. Labels - tags for finding resources

```
kubectl get pods --show-labels
kubectl get pods -l app=nginx
kubectl get pods -l environment=dev
kubectl label pod nginx-pod environment=production
```

The selector `-l app=nginx` filters Pods by label. The final command removes the environment label. Labels are vital because Services and controllers use selectors to manage Pods.

8. Practice tasks

- **Task 1:** Create the Nginx Pod from YAML. Verify it is Running and inspect it.
- **Task 2:** Enter the Nginx container and request `localhost:80`.
- **Task 3:** Create the BusyBox Pod and read its logs.
- **Task 4:** Create a Pod imperatively, then export its YAML. Compare with hand-written YAML.
- **Task 5:** Add, query, and remove labels. Record the exact output.
- **Task 6:** Delete the Nginx Pod and recreate it. Notice standalone Pods are not automatically recreated.

9. Cleanup

```
kubectl delete pod nginx-pod busybox-pod redis-pod test-pod --ignore-not-found
kubectl delete -f nginx-pod.yaml
```

Important: A standalone Pod does not provide deployment-style self-healing. In the next day, a Deployment will manage replacement Pods.

DAY 52

Namespaces & Deployments

Goal: Organize resources with namespaces and manage replicated, self-healing application Pods with Deployments.

1. Namespaces

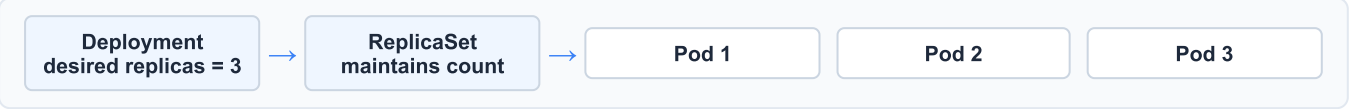
A namespace is a logical grouping boundary inside a cluster. It helps separate teams, environments, and application resources. It is not a separate cluster and not a complete security boundary by itself.

Namespace	Typical purpose
`default`	Namespace used when you do not specify another one.
`kube-system`	Kubernetes system components.
`kube-public`	Readable by all users by default; rarely used for app workloads.
`kube-node-lease`	Node heartbeat/lease objects.
`dev / staging / production`	Example namespaces for application environments.

```
kubectl get namespaces
kubectl create namespace dev
kubectl create namespace staging
```

2. What is a Deployment?

A Deployment manages a set of identical Pods. You declare how many replicas you want. Kubernetes creates and replaces Pods to move the actual number toward that desired number.



A Deployment manages a ReplicaSet; the ReplicaSet manages the Pods. You normally change the Deployment, not individual Pods.

3. Deployment YAML - field-by-field

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: dev
```

Field	Plain-English meaning
`apiVersion: apps/v1` & `kind: Deployment`	API group/version and resource type.
`metadata.namespace`	Create it in the `dev` namespace.
`spec.replicas: 3`	Keep three matching Pods running.
`spec.selector.matchLabels` & `template`	Find Pods matching `app=nginx`; template is the blueprint for each Pod.

Critical matching rule: `spec.selector.matchLabels` must match the labels under `spec.template.metadata.labels`.

4. Apply, Inspect & 5. Scaling

```
kubectl apply -f nginx-deployment.yaml
kubectl get deployments,replicasets,pods -n dev -o wide
kubectl scale deployment nginx-deployment --replicas=5 -n dev
```

6. Rolling update and rollback

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.25 -n dev
kubectl rollout status deployment/nginx-deployment -n dev
kubectl rollout history deployment/nginx-deployment -n dev
```

The first command changes the image for the container named nginx. A rolling update gradually replaces old Pods with new ones. Rollout status watches progress; history shows revisions; undo returns to the previous revision.

7. Self-healing demonstration

- Delete one Pod managed by the Deployment: `kubectl delete pod <pod-name> -n dev`.
- Immediately run `kubectl get pods -n dev` and watch for a replacement.
- Explain why this is different from the standalone Pod from Day 51.

8. Practice tasks

- **Task 1:** Create dev, staging, and production namespaces.
- **Task 2:** Deploy three Nginx replicas into dev using the manifest.
- **Task 3:** Scale to five and then back to two.
- **Task 4:** Delete one Pod and observe its replacement.
- **Task 5:** Update the image, wait for rollout, view history, and undo.

9. Cleanup

```
kubectl delete deployment nginx-deployment -n dev --ignore-not-found
kubectl delete namespace dev staging production
```

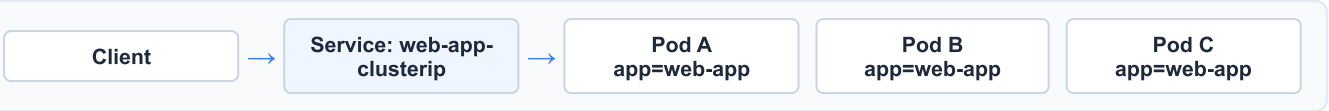
DAY 53

Kubernetes Services

Goal: Give changing Pods a stable network endpoint and understand ClusterIP, NodePort, and LoadBalancer Services.

1. Why do we need a Service?

Pods can be replaced. A replacement Pod can have a different IP address. If clients connect directly to a Pod IP, they may lose connection when the Pod is replaced. A Service provides a stable virtual IP and DNS name and forwards traffic to matching backend Pods.



2. Deploy the application

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
kubectl apply -f web-app-deployment.yaml
```

3. ClusterIP (internal cluster access)

ClusterIP is the default Service type, exposing the app on a stable virtual IP reachable within the cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: web-app-clusterip
```

Field	Meaning
`type: ClusterIP`	Create an internal cluster Service.
`port: 80` / `targetPort: 80`	Client-facing port vs backend container port.

```
kubectl apply -f web-app-clusterip.yaml
kubectl get services,endpoints web-app-clusterip
```

4. NodePort (expose through a node port)

NodePort opens a port on each node (range 30000-32767) and forwards traffic to the Service.

```
apiVersion: v1
kind: Service
metadata:
```

5. Load Balancer

Requests an external load balancer from cloud providers. On local clusters, EXTERNAL-IP may remain Pending.

```
apiVersion: v1
kind: Service
```

6. Compare Service types

Type	Typical reachability	Use case
ClusterIP	Inside the cluster	Internal app-to-app communication.
NodePort	Through a node IP and allocated port	Basic external access or learning.
LoadBalancer	External address provided by infrastructure	Cloud-facing service with provider load balancer.

7. Practice tasks

- **Task 1:** Deploy three Nginx Pods using a Deployment with label `app=web-app`.
- **Task 2:** Create a ClusterIP Service and inspect its endpoints.
- **Task 3:** Use a temporary BusyBox Pod to fetch the Service DNS name.
- **Task 4:** Create a NodePort Service on 30080 and test access.
- **Task 5:** Create a LoadBalancer Service and observe external IP status.

8. Troubleshooting & 9. Cleanup

Problem	Likely cause / check
Service has no endpoints	Selector does not match Pod labels; Pods may not be Ready.
DNS name fails	Check Service name, namespace, and CoreDNS Pods.
LoadBalancer EXTERNAL-IP Pending	No external or local load-balancer implementation is available.

```
kubectl delete service web-app-clusterip web-app-nodeport web-app-lb --ignore-not-found
kubectl delete deployment web-app --ignore-not-found
```

DAY 54

ConfigMaps & Secrets

Goal: Keep application configuration outside the container image and distinguish ordinary configuration from sensitive credentials.

1. Configuration vs application image

An application image contains the program and its dependencies. Configuration controls how the program behaves in a particular environment (environment name, port, feature flags, database connection settings). Separating configuration means the same image can be used across environments.



2. ConfigMap (non-sensitive settings)

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_DEBUG=false \
```

3. Create a ConfigMap from a file

Create a file named `default.conf` with an Nginx server block where `/health` responds with `healthy`.

```
server {
    listen 80;
    server_name localhost;
    location /health { return 200 "healthy\n"; }
}
```

```
kubectl create configmap nginx-config --from-file=default.conf=./default.conf
kubectl describe configmap nginx-config
```

4. Use ConfigMap values as environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: config-env-pod
```

```
kubectl apply -f config-env-pod.yaml && kubectl logs config-env-pod
```

5. Mount a ConfigMap as a volume

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-config-pod
```

6. Secret (intended for sensitive values)

A Secret stores sensitive data like passwords and tokens. Kubernetes Secrets are Base64-encoded in YAML. Base64 is encoding, not encryption.

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD='lab-password'
```

8. Inject a Secret as an environment variable & 9. Mount as files

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
```

10. Live ConfigMap updates

A ConfigMap mounted as a normal volume can be updated by Kubernetes after a short delay. Environment variables do not change inside already-running processes.

```
kubectl create configmap live-config --from-literal=message=hello
kubectl patch configmap live-config --type merge -p '{"data":{"message":"world"}}'
```

11. Practice tasks

- **Task 1:** Create app-config with APP_ENV, APP_DEBUG, and APP_PORT.
- **Task 2:** Create default.conf with /health and store in nginx-config.
- **Task 3 & 4:** Inject ConfigMap values into BusyBox and mount Nginx config.
- **Task 5 & 6:** Create db-credentials, explain Base64, and inject via secretKeyRef.
- **Task 7:** Update live-config and observe mounted file changes.

12. Cleanup

```
kubectl delete pod config-env-pod nginx-config-pod secret-env-pod --ignore-not-found
kubectl delete configmap app-config nginx-config live-config --ignore-not-found
```

Security reminder: Do not commit real Secret manifests containing credentials to Git. Combine Secret management with RBAC, encryption at rest, and audit logging.

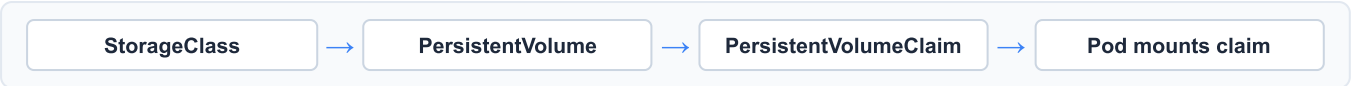
DAY 55

Persistent Volumes & Persistent Volume Claims

Goal: Understand temporary vs persistent storage, create manual PV/PVC, and use dynamic provisioning.

Term	Simple definition
Volume	A storage location mounted into a container.
emptyDir	Temporary Pod-scoped storage.
PV (PersistentVolume)	A cluster-level representation of storage capacity.
PVC (PersistentVolumeClaim)	A namespaced request for storage by an application.
StorageClass	A named storage type/provisioning configuration.

1. Storage Object Relationship & Task 1 (emptyDir)



Task 1: Prove emptyDir is temporary

```
apiVersion: v1
kind: Pod
metadata:
```

2. Task 2: Create a manual PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

3. Task 3: Request storage with a PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

```
kubectl apply -f manual-pvc.yaml && kubectl get pvc,pv
```

A PVC is a request, not the disk itself. This claim requests at least 500Mi and asks for a classless PV. The 1Gi PV satisfies the request.

4. Task 4: Mount the PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: persistent-pod
```

5. Task 5: Inspect StorageClasses & 6. Dynamic Provisioning

```
kubectl get storageclass
kubectl describe storageclass standard
```

Setting	Meaning
Provisioner	Driver/controller that creates the storage.
ReclaimPolicy: Delete / Retain	Delete backing storage upon claim deletion vs keep for administrator recovery.
WaitForFirstConsumer	Waits until a Pod uses the claim before provisioning, helping topology placement.

Dynamic PVC manifest:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

7. Static vs dynamic provisioning

Static provisioning	Dynamic provisioning
Administrator creates a PV first.	PVC + StorageClass trigger PV creation.
Manually match claim requirements to an existing PV.	Provisioner creates storage satisfying the claim.

8. Cleanup & Reclaim Behavior

```
kubectl delete pod emptydir-pod persistent-pod dynamic-pod --ignore-not-found
kubectl delete pvc manual-pvc dynamic-pvc
```

9. Access modes nuance

Access Mode	Meaning in simple terms
ReadWriteOnce (RWO)	Read/write mount from one node at a time.
ReadOnlyMany (ROX)	Read-only mounts from many nodes.
ReadWriteMany (RWX)	Read/write mounts from many nodes.

10. Troubleshooting Storage

Problem	Checks
PVC Pending	Check events, class name, capacity, access modes, and whether a Pod is needed for WaitForFirstConsumer.
PVC cannot find manual PV	Verify storageClassName, access mode compatibility, Available status, and capacity.
Pod stuck ContainerCreating	Inspect volume mount and attach events.

Cross-Day Revision: A Simple Web App in Kubernetes

Imagine deploying a web application: put it in a **Namespace**; use a **Deployment** to keep multiple Pods running; expose them through a **Service**; inject runtime settings through a **ConfigMap** and **Secret**; and mount a **PVC** when data needs to survive Pod replacement.

Most important kubectl commands

Command	Use
`kubectl get`	List resources.
`kubectl describe`	Inspect details, conditions, and events.
`kubectl apply -f file.yaml`	Create or update resources from YAML.
`kubectl logs`	Read container logs.
`kubectl exec -it ... -- sh`	Open a shell inside a container.
`kubectl get pods -A`	List Pods across all namespaces.

Vocabulary flash cards

Word	Remember it as...
Cluster / Node	The whole Kubernetes environment / A machine in the cluster.
Control plane / Pod	Decision & API layer / Deployable wrapper around container(s).
Manifest / Label	YAML description of an object / Key-value metadata tag.
Deployment / ReplicaSet	Controller for replicated Pods / Maintains desired Pod count.
Service / ConfigMap	Stable network access / Non-sensitive configuration data.

Final Hands-On Project - Deploy a Configurable, Persistent Web App

Use this mini-project to prove your understanding across all six days using Nginx.

Project requirements

- Create a namespace named `practice-app`.
- Create a Deployment with two Nginx replicas and labels `app=practice-web`.
- Create a ClusterIP Service selecting `app=practice-web`.
- Create a ConfigMap containing a custom Nginx health endpoint and mount it.
- Create a PVC using an available StorageClass and mount it into a BusyBox Pod to write a file.

Suggested execution order

Step	Action	Verify
1 & 2	Create namespace `practice-app` & Apply Deployment YAML	`kubectl get deploy,pods -n practice-app`
3 & 4	Apply ClusterIP Service & Create/mount ConfigMap	`kubectl get svc,endpoints` / inspect file
5 & 6	Create PVC & Mount claim in Pod, write test data	`kubectl exec ... cat /data/test.txt`
7 & 8	Delete/recreate Pod & Clean up resources	Verify file remains & delete namespace

Self-assessment checklist

Skill	I can do it	Need practice
Explain control plane and worker-node components	[ ]	[ ]
Create and inspect a Pod from YAML	[ ]	[ ]
Use labels and selectors	[ ]	[ ]
Create namespaces and Deployments	[ ]	[ ]
Scale, update, and roll back a Deployment	[ ]	[ ]
Create ClusterIP, NodePort, and LoadBalancer Services	[ ]	[ ]
Explain Service port vs targetPort	[ ]	[ ]
Create and mount a ConfigMap	[ ]	[ ]
Create and use a Secret safely	[ ]	[ ]
Explain Base64 vs encryption	[ ]	[ ]
Explain emptyDir vs persistent storage	[ ]	[ ]
Create a PV and PVC	[ ]	[ ]
Use dynamic provisioning	[ ]	[ ]
Troubleshoot Pending Pods and PVCs using events	[ ]	[ ]

Revision Quiz: Test Yourself

Try answering without looking at notes, then check the answer key on the next page.

1. Which component receives kubectl API requests?
2. Which component stores Kubernetes cluster state?
3. What is the smallest deployable Kubernetes unit?
4. Why do we use labels and selectors?
5. What is the relationship between a Deployment, ReplicaSet, and Pods?
6. Does containerPort expose a Pod to the internet?
7. What problem does a Service solve?
8. What is the difference between port and targetPort in a Service?
9. Which Service type is normally internal to the cluster?
0. What kind of data belongs in a ConfigMap?
1. Is Base64 the same as encryption?
2. Do environment variables automatically change when a ConfigMap changes?
3. Does emptyDir survive deletion and recreation of the Pod?
4. What is the difference between PV and PVC?
5. What does WaitForFirstConsumer mean?

Answer Key

#	Answer
1	API Server.
2	etcd.
3	Pod.
4	Labels tag objects; selectors find objects with matching labels.
5	Deployment manages a ReplicaSet; ReplicaSet maintains desired Pod count.
6	No. It documents container ports; a Service or exposure method is needed.
7	It gives stable network access to changing/replaced Pods.
8	port is Service-facing; targetPort is backend Pod/container port.
9	ClusterIP.
10	Non-sensitive configuration.
11	No. Base64 is encoding, not encryption.
12	No. Environment variables are set when the container starts.
13	No. It is removed with the Pod.
14	PV represents storage; PVC is a namespaced request for storage.
15	Binding/provisioning waits until a Pod uses the claim for topology consideration.
16	No. RWO means one node at a time, not necessarily one Pod.

Reusable Daily Documentation Template

```
# Day XX Topic
## Goal: What am I learning today?
## Concepts in my own words (Term / Meaning / Why it matters)
## Architecture / flow: Describe components and communication.
## Files created: file-name.yaml
## Commands I ran / Expected result / Actual result
## Troubleshooting (Symptom / Evidence / Root cause / Fix)
## Practice completed & What I learned
```

Good practice habits

- Use versioned images instead of latest for repeatable exercises.
- Use `kubectl describe` and `kubectl get events` when something fails.
- Check resource namespaces (Pods, Deployments, Services, ConfigMaps, Secrets, PVCs are namespaced; PVs and StorageClasses are cluster-scoped).
- Clean up lab resources so old workloads do not confuse the next exercise.
- Never put real credentials into Git, screenshots, or public documentation.

Final Learning Advice

> **Repeat each lab at least twice.**
> First time, follow the guide. Second time, close the guide and recreate the resources from memory. You are ready to move on when you can explain what each YAML field does and diagnose a failure using kubectl.

Summary of Covered Topics (Days 50-55)

- **Day 50:** Control plane components (API Server, etcd, Scheduler, Controller Manager), worker node components (kubelet, kube-proxy, container runtime), and cluster setup.
- **Day 51:** Pod manifests, YAML structure, logs, interactive container shell access, and labels/selectors.
- **Day 52:** Namespaces, Deployments, ReplicaSets, scaling, rolling updates, and self-healing.
- **Day 53:** Services (ClusterIP, NodePort, LoadBalancer) and stable networking.
- **Day 54:** ConfigMaps, Secrets, environment variable injection, and volume mounts.
- **Day 55:** Persistent storage, emptyDir, PersistentVolumes (PV), PersistentVolumeClaims (PVC), StorageClasses, and dynamic provisioning.

End of Days 50-55 Beginner Workbook